

Création d'un Framework symfony

*Une synthèse étape par étape basé sur la documentation
Symfony*

yatlas

18 février 2026

Table des matières

1	Introduction	3
1.1	Étape 1 : Les bases	3
1.2	Cycle de Vie de la Requête	3
2	Le composant HttpFoundation	5
2.1	Installation	5
2.2	Refactoring de l'application	5
2.3	Cycle de Vie de la Requête	6
3	Le Contrôleur Frontal	7
3.1	Première tentative de refactoring (init.php)	7
3.2	Restructuration du projet	7
3.3	Le Contrôleur Frontal (front.php)	8
3.4	Les Templates	8
3.5	Cycle de Vie de la Requête	9
4	Le composant Routing	10
4.1	Installation	10
4.2	Configuration des routes (src/app.php)	10
4.3	Mise à jour du Contrôleur Frontal	10
4.4	Simplification des templates	11
4.5	Cycle de Vie de la Requête	11
5	Le système de Templating (Contrôleurs)	13
5.1	Introduction de l'attribut <code>_controller</code>	13
5.2	Nouvelle application : Année Bissextile	13
5.3	Cycle de Vie de la Requête	14
6	Le composant HttpKernel (Le Résolveur de Contrôleur)	15
6.1	Installation	15
6.2	Création du Contrôleur (src/LeapYearController.php)	15
6.3	Utilisation des Résolveurs (web/front.php)	16
6.4	Cycle de Vie de la Requête	16
7	La Séparation des Préoccupations	18
7.1	Création de la classe Framework (Simplex)	18
7.2	Organisation de l'Application (Calendar)	19
7.3	Le Contrôleur Frontal Simplifié	19
7.4	Cycle de Vie de la Requête	19

8 Les Tests Unitaires	21
8.1 Installation et Configuration	21
8.2 Refactoring pour la Testabilité	21
8.2.1 Pourquoi utiliser des Interfaces ?	22
8.3 Nos premiers tests (FrameworkTest)	22
9 Le Composant EventDispatcher	23
9.1 Installation	23
9.2 Mise en place des événements	23
9.3 Création de Subscribers	23
9.4 Cycle de Vie de la Requête (Final)	24
10 Le Composant HttpKernel (HttpKernelInterface)	26
10.1 Implémentation de l'Interface	26
10.2 Le Cache HTTP	26
10.3 Test du Cache	27
10.4 Cycle de Vie de la Requête	27
11 Le Composant HttpKernel (La Classe HttpKernel)	28
11.1 Le Framework Simplifié	28
11.2 Mise à jour du Contrôleur Frontal	28
11.3 Gestion des Erreurs (ErrorListener)	28
11.4 Préparation de la Réponse (ResponseListener)	29
11.5 Le StringResponseListener	29
11.6 Cycle de Vie de la Requête (Final)	29
12 L'Injection de Dépendances	31
12.1 Installation	31
12.2 Configuration du Conteneur (src/container.php)	31
12.3 Le Contrôleur Frontal Nettoyé	31

Chapitre 1

Introduction

Ce rapport documente le processus de construction d'un framework web PHP moderne, en suivant le guide "Create your own framework" de Symfony. Nous allons implémenter le framework étape par étape, en expliquant chaque ajout et son utilité.

1.1 Étape 1 : Les bases

Pour commencer, nous avons mis en place la structure initiale du projet. L'objectif est de créer une application web simple sans utiliser de bibliothèques externes pour le moment, afin de comprendre les fondements.

Nous avons créé le dossier `application` qui contiendra notre code source. À l'intérieur, nous avons créé un fichier `index.php` qui agit comme le point d'entrée de notre application.

Voici le code initial de notre fichier `index.php` :

```
1 <?php
2
3 $name = $_POST['name'] ?? $_GET['name'] ?? 'World';
4
5 header('Content-Type: text/html; charset=utf-8');
6
7 printf('Hello %s', htmlspecialchars($name, ENT_QUOTES, 'UTF-8'));
```

Listing 1.1 – Code initial de index.php

Ce script récupère un paramètre `name` depuis la requête HTTP (GET ou POST). Si le paramètre n'est pas fourni, il utilise "World" comme valeur par défaut. Ensuite, il définit l'en-tête de réponse pour indiquer que le contenu est du HTML encodé en UTF-8, et enfin, il affiche un message de salutation en échappant les caractères spéciaux pour éviter les failles XSS.

Cette première étape nous permet de valider que notre environnement PHP fonctionne correctement et que nous pouvons traiter des requêtes HTTP basiques.

1.2 Cycle de Vie de la Requête

1. Le navigateur envoie une requête HTTP à `index.php`.
2. Le script récupère les globales PHP (`$_GET`, `$_POST`).
3. Il traite la logique (valeur par défaut).

-
4. Il envoie les en-têtes HTTP et le corps de la réponse via `printf`.

Chapitre 2

Le composant HttpFoundation

La deuxième étape consiste à introduire une approche orientée objet pour gérer les requêtes et les réponses HTTP, en remplaçant les variables globales de PHP (`$_GET`, `$_POST`, etc.). Pour cela, nous utilisons le composant `symfony/http-foundation`.

2.1 Installation

Nous avons installé le composant via Composer avec la commande suivante :

```
1 composer require symfony/http-foundation
```

Listing 2.1 – Installation de HttpFoundation

Cela a également généré le dossier `vendor` et le fichier `autoload.php` pour le chargement automatique des classes.

2.2 Refactoring de l'application

Nous avons ensuite modifié le fichier `index.php` pour utiliser les classes `Request` et `Response` fournies par le composant.

```
1 <?php
2
3 require_once __DIR__ . '/vendor/autoload.php';
4
5 use Symfony\Component\HttpFoundation\Request;
6 use Symfony\Component\HttpFoundation\Response;
7
8 $request = Request::createFromGlobals();
9
10 $name = $request->query->get('name', 'World');
11
12 $response = new Response(sprintf('Hello %s', htmlspecialchars($name
13     , ENT_QUOTES, 'UTF-8')));
14 $response->send();
```

Listing 2.2 – index.php avec HttpFoundation

Avec la méthode `createFromGlobals()`, nous créons un objet `Request` qui encapsule les variables globales. Nous récupérons ensuite le paramètre `name` via `$request->query->get()`. Enfin, nous construisons un objet `Response` avec le contenu HTML et nous l'envoyons au client avec la méthode `send()`.

Cette approche offre une meilleure abstraction et un contrôle plus fin sur les messages HTTP, tout en sécurisant davantage l'application (par exemple pour la récupération des adresses IP via des proxys).

2.3 Cycle de Vie de la Requête

1. Le navigateur envoie une requête à `index.php`.
2. `Request::createFromGlobals()` capture les données de la requête dans un objet.
3. L'application lit les données via l'objet `$request`.
4. Un objet `Response` est instancié avec le contenu.
5. `$response->send()` envoie les en-têtes et le contenu au navigateur.

Chapitre 3

Le Contrôleur Frontal

La troisième étape introduit le modèle "Front Controller". Jusqu'à présent, chaque page (bien que nous n'en ayons qu'une) aurait nécessité son propre script PHP accessible publiquement. Le "Front Controller" est un point d'entrée unique pour toutes les requêtes de l'application, ce qui permet de centraliser la configuration et la gestion des requêtes.

3.1 Première tentative de refactoring (init.php)

Avant d'arriver à la solution finale, la documentation suggère une première approche pour éviter la duplication de code entre les différentes pages : l'utilisation d'un fichier `init.php`.

Nous avons considéré l'extraction de l'initialisation de la `Request` et de la `Response` dans un fichier commun :

```
1 // framework/init.php
2 require_once __DIR__ . '/vendor/autoload.php';
3 use Symfony\Component\HttpFoundation\Request;
4 use Symfony\Component\HttpFoundation\Response;
5
6 $request = Request::createFromGlobals();
7 $response = new Response();
```

Listing 3.1 – Refactoring intermédiaire (init.php)

Bien que cela réduise la duplication, cette solution n'est pas idéale car elle conserve la méthode `send()` dans chaque page et ne résout pas le problème de l'exposition directe des scripts PHP aux URL. C'est pourquoi nous avons opté pour le modèle "Front Controller".

3.2 Restructuration du projet

Nous avons réorganisé la structure du projet pour séparer clairement les fichiers accessibles publiquement de la logique interne :

- `web/` : Le dossier public contenant le contrôleur frontal (`front.php`).
- `src/pages/` : Le dossier contenant les modèles (templates) de nos pages.

3.3 Le Contrôleur Frontal (front.php)

Le fichier `web/front.php` gère désormais toutes les requêtes. Il utilise un système de mappage simple pour associer les URL aux fichiers de template correspondants.

```
1 <?php
2
3 require_once __DIR__.'../../vendor/autoload.php';
4
5 use Symfony\Component\HttpFoundation\Request;
6 use Symfony\Component\HttpFoundation\Response;
7
8 $request = Request::createFromGlobals();
9 $response = new Response();
10
11 $map = [
12     '/hello' => __DIR__.'../../src/pages/hello.php',
13     '/bye'   => __DIR__.'../../src/pages/bye.php',
14 ];
15
16 $path = $request->getPathInfo();
17 if (isset($map[$path])) {
18     ob_start();
19     include $map[$path];
20     $response->setContent(ob_get_clean());
21 } else {
22     $response->setStatusCode(404);
23     $response->setContent('Not Found');
24 }
25
26 $response->send();
```

Listing 3.2 – Le Contrôleur Frontal (web/front.php)

3.4 Les Templates

Les fichiers dans `src/pages/` ne contiennent plus la logique d'initialisation de la requête ou de l'envoi de la réponse. Ils se contentent d'afficher le contenu HTML, qui est capturé par le contrôleur frontal grâce aux fonctions de bufferisation de sortie (`ob_start()` et `ob_get_clean()`).

Exemple de `src/pages/hello.php` :

```
1 <!-- example.com/src/pages/hello.php -->
2 <?php $name = $request->query->get('name', 'World') ?>
3 Hello <?= htmlspecialchars($name, ENT_QUOTES, 'UTF-8') ?>
```

Listing 3.3 – Template Hello (src/pages/hello.php)

Cette architecture permet des URL plus propres (toutes gérées par `front.php`) et une meilleure séparation des responsabilités.

3.5 Cycle de Vie de la Requête

1. Le navigateur envoie une requête (ex : `/hello`).
2. `front.php` (Contrôleur Frontal) reçoit la requête.
3. Il compare le chemin de la requête avec son tableau de mappage (`$map`).
4. Il inclut le fichier template correspondant en activant la bufferisation de sortie.
5. Le contenu capturé est mis dans un objet **Response**.
6. La réponse est envoyée au client.

Chapitre 4

Le composant Routing

L'étape 4 se concentre sur l'organisation des URL. Jusqu'à présent, nous utilisons un simple tableau associatif pour mapper les URL aux templates. Cependant, cette méthode est limitée (par exemple, elle ne gère pas les paramètres dynamiques comme `/hello/Fabien`).

Nous avons donc introduit le composant `symfony/routing`.

4.1 Installation

Comme pour `HttpFoundation`, l'installation se fait via `Composer` :

```
1 composer require symfony/routing
```

Listing 4.1 – Installation de Routing

4.2 Configuration des routes (`src/app.php`)

Nous avons déplacé la configuration des routes dans un fichier dédié `src/app.php`, en utilisant l'objet `RouteCollection` :

```
1 <?php
2 // example.com/src/app.php
3 use Symfony\Component\Routing;
4
5 $routes = new Routing\RouteCollection();
6 $routes->add('hello', new Routing\Route('/hello/{name}', ['name' =>
7     'World']));
8
9 $routes->add('bye', new Routing\Route('/bye'));
10
11 return $routes;
```

Listing 4.2 – Configuration des routes (`src/app.php`)

Notez l'utilisation de `{name}` qui définit un paramètre dynamique dans l'URL.

4.3 Mise à jour du Contrôleur Frontal

Le fichier `web/front.php` a été mis à jour pour utiliser `UrlMatcher` afin de faire correspondre la requête entrante avec les routes définies.

```
1 <?php
2 // example.com/web/front.php
3 require_once __DIR__.'../vendor/autoload.php';
4
5 use Symfony\Component\HttpFoundation\Request;
6 use Symfony\Component\HttpFoundation\Response;
7 use Symfony\Component\Routing;
8
9 $request = Request::createFromGlobals();
10 $routes = include __DIR__.'../src/app.php';
11
12 $context = new Routing\RequestContext();
13 $context->fromRequest($request);
14 $matcher = new Routing\Matcher\UrlMatcher($routes, $context);
15
16 try {
17     extract($matcher->match($request->getPathInfo()), EXTR_SKIP);
18     ob_start();
19     include sprintf(__DIR__.'../src/pages/%s.php', $_route);
20     $response = new Response(ob_get_clean());
21 } catch (Routing\Exception\ResourceNotFoundException $exception) {
22     $response = new Response('Not Found', 404);
23 } catch (Exception $exception) {
24     $response = new Response('An error occurred', 500);
25 }
26
27 $response->send();
```

Listing 4.3 – Contrôleur Frontal avec Routing (web/front.php)

4.4 Simplification des templates

Grâce à la fonction `extract()`, les paramètres de route (comme `$name`) sont directement accessibles dans les templates, ce qui simplifie leur code :

```
1 <!-- example.com/src/pages/hello.php -->
2 Hello <?= htmlspecialchars($name, ENT_QUOTES, 'UTF-8') ?>
```

Listing 4.4 – Template Hello simplifié (src/pages/hello.php)

Nous avons maintenant un système de routage puissant et flexible.

4.5 Cycle de Vie de la Requête

1. Le navigateur envoie une requête (ex : `/hello/Fabien`).
2. `front.php` initialise le contexte de routage et le `UrlMatcher`.
3. `$matcher->match()` analyse l'URL et retourne les attributs (y compris `_route` et les paramètres).
4. `extract()` transforme ces attributs en variables.

-
5. Le template est inclus dynamiquement selon la route correspondante.
 6. La réponse est générée et envoyée.

Chapitre 5

Le système de Templating (Contrôleurs)

L'étape 5 vise à séparer davantage la logique de l'affichage en introduisant la notion de **Contrôleur**.

Jusqu'à présent, le rendu des templates était "codé en dur" dans le contrôleur frontal. Nous voulons rendre cela dynamique pour que chaque route puisse définir sa propre logique.

5.1 Introduction de l'attribut `_controller`

Nous avons modifié le fichier `web/front.php` pour qu'il ne décide plus lui-même quel fichier inclure, mais qu'il exécute une fonction (ou un "callable") définie dans les attributs de la route, sous la clé `_controller`.

Une fonction générique `render_template` a été créée pour conserver la fonctionnalité précédente (le rendu simple de templates) :

```
1 function render_template(Request $request): Response
2 {
3     extract($request->attributes->all(), EXTR_SKIP);
4     ob_start();
5     include sprintf(__DIR__.'../../src/pages/%s.php', $_route);
6
7     return new Response(ob_get_clean());
8 }
```

Listing 5.1 – Fonction `render_template` (`web/front.php`)

Le contrôleur frontal utilise ensuite `call_user_func` pour exécuter le contrôleur approprié :

```
1 $request->attributes->add($matcher->match($request->getPathInfo()))
2 ;
3 $response = call_user_func($request->attributes->get('_controller'),
4     $request);
```

5.2 Nouvelle application : Année Bissextile

Pour démontrer la flexibilité de cette approche, nous avons remplacé notre application "Hello World" par une application déterminant si une année est bissextile.

Dans `src/app.php`, nous avons défini une fonction `is_leap_year` et une route utilisant une `Closure` comme contrôleur :

```
1 // ...
2 function is_leap_year(?int $year = null): bool
3 {
4     if (null === $year) {
5         $year = (int) date('Y');
6     }
7
8     return 0 === $year % 400 || (0 === $year % 4 && 0 !== $year %
9         100);
10 }
11 $routes->add('leap_year', new Routing\Route('/is_leap_year/{year}',
12     [
13         'year' => null,
14         '_controller' => function (Request $request): Response {
15             if (is_leap_year($request->attributes->get('year'))) {
16                 return new Response('Yep, this is a leap year!');
17             }
18             return new Response('Nope, this is not a leap year.');
```

Listing 5.2 – Application "Année Bissextille" (`src/app.php`)

Cette étape marque la naissance d'un véritable framework capable de déléguer la logique à des contrôleurs spécifiques.

5.3 Cycle de Vie de la Requête

1. Le navigateur envoie une requête.
2. `UrlMatcher` retourne les attributs, dont `_controller` (une Closure ici).
3. Le script ajoute ces attributs à l'objet `$request`.
4. `call_user_func()` exécute la Closure trouvée.
5. La Closure retourne un objet `Response`.
6. La réponse est envoyée.

Chapitre 6

Le composant HttpKernel (Le Résolveur de Contrôleur)

Bien que fonctionnelle, l'approche précédente avec les Closures a un inconvénient : toutes les routes sont chargées et instanciées à chaque requête. Pour améliorer cela, nous allons utiliser de vraies classes de contrôleur.

L'étape 6 introduit le composant `HttpKernel` pour gérer la résolution des contrôleurs et de leurs arguments.

6.1 Installation

Nous avons d'abord configuré l'autoloading PSR-4 dans `composer.json` pour charger nos classes depuis le dossier `src/` :

```
1 "autoload": {  
2     "psr-4": {  
3         "": "src/"  
4     }  
5 }
```

Puis nous avons installé le composant :

```
1 composer require symfony/http-kernel
```

6.2 Création du Contrôleur (src/LeapYearController.php)

Nous avons migré la logique de l'année bissextile dans une classe dédiée :

```
1 <?php  
2 use Symfony\Component\HttpFoundation\Request;  
3 use Symfony\Component\HttpFoundation\Response;  
4  
5 class LeapYearController  
6 {  
7     public function index(int $year): Response  
8     {  
9         if (is_leap_year($year)) {  
10             return new Response('Yep, this is a leap year!');
```



```
11     }
12
13     return new Response('Nope, this is not a leap year.');
```

Listing 6.1 – LeapYearController (src/LeapYearController.php)

Dans `src/app.php`, la route pointe désormais vers cette classe :

```
1 $routes->add('leap_year', new Routing\Route('/is_leap_year/{year}',
2     ['year' => null,
3     '_controller' => 'LeapYearController::index',
4 ]));
```

6.3 Utilisation des Résolveurs (web/front.php)

Enfin, nous avons mis à jour le contrôleur frontal pour utiliser `ControllerResolver` et `ArgumentResolver`. Ces classes se chargent d'instancier le contrôleur et d'injecter les bons arguments (comme `$year` ou `$request`) automatiquement.

```
1 // ...
2 use Symfony\Component\HttpKernel;
3
4 $controllerResolver = new HttpKernel\Controller\ControllerResolver
5     ();
6 $argumentResolver = new HttpKernel\Controller\ArgumentResolver();
7
8 try {
9     $request->attributes->add($matcher->match($request->getPathInfo
10         ()));
11
12     $controller = $controllerResolver->getController($request);
13     $arguments = $argumentResolver->getArguments($request,
14         $controller);
15
16     $response = call_user_func_array($controller, $arguments);
17 } catch (Routing\Exception\ResourceNotFoundException $exception) {
18     // ...
```

Listing 6.2 – Contrôleur Frontal avec HttpKernel (web/front.php)

Notre framework supporte maintenant les contrôleurs sous forme de classes avec injection d'arguments !

6.4 Cycle de Vie de la Requête

1. `UrlMatcher` identifie la route et retourne `LeapYearController::index` comme contrôleur.
2. `ControllerResolver` instancie la classe `LeapYearController`.

-
3. **ArgumentResolver** analyse la méthode `index($year)`, voit qu'elle a besoin de `$year`, et récupère la valeur depuis les attributs de requête.
 4. Le contrôleur est exécuté avec les bons arguments.
 5. Le contrôleur retourne une **Response**.

Chapitre 7

La Séparation des Préoccupations

Notre fichier `front.php` commence à contenir trop de logique. Il est temps de séparer le code du framework (générique) du code de l'application (spécifique).

L'étape 7 consiste à créer une classe `Framework` réutilisable et à organiser notre application "Calendar" proprement.

7.1 Création de la classe Framework (Simplex)

Nous avons créé un nouvel espace de noms `Simplex` et une classe `Framework` qui encapsule la logique de traitement de la requête :

```
1 <?php
2 namespace Simplex;
3
4 use Symfony\Component\HttpFoundation\Request;
5 use Symfony\Component\HttpFoundation\Response;
6 use Symfony\Component\HttpKernel\Controller\ArgumentResolver;
7 use Symfony\Component\HttpKernel\Controller\ControllerResolver;
8 use Symfony\Component\Routing\Exception\ResourceNotFoundException;
9 use Symfony\Component\Routing\Matcher\UrlMatcher;
10
11 class Framework
12 {
13     public function __construct(
14         private UrlMatcher $matcher,
15         private ControllerResolver $controllerResolver,
16         private ArgumentResolver $argumentResolver,
17     ) {
18     }
19
20     public function handle(Request $request): Response
21     {
22         $this->matcher->getContext()->fromRequest($request);
23
24         try {
25             $request->attributes->add($this->matcher->match(
26                 $request->getPathInfo()));
```

```
27         $controller = $this->controllerResolver->getController(  
28             $request);  
29         $arguments = $this->argumentResolver->getArguments(  
30             $request, $controller);  
31         return call_user_func_array($controller, $arguments);  
32     } catch (ResourceNotFoundException $exception) {  
33         return new Response('Not Found', 404);  
34     } catch (\Exception $exception) {  
35         return new Response('An error occurred', 500);  
36     }  
37 }
```

Listing 7.1 – La classe Framework (src/Simplex/Framework.php)

7.2 Organisation de l'Application (Calendar)

Nous avons déplacé la logique de l'application dans l'espace de noms `Calendar`, en séparant le Modèle et le Contrôleur :

- `src/Calendar/Model/LeapYear.php` : Contient la logique métier `isLeapYear`.
- `src/Calendar/Controller/LeapYearController.php` : Le contrôleur qui utilise le modèle.

7.3 Le Contrôleur Frontal Simplifié

Grâce à ces changements, notre fichier `web/front.php` est devenu très concis. Il se contente d'initialiser le framework et de lui passer la requête :

```
1 // ...  
2 $framework = new Simplex\Framework($matcher, $controllerResolver,  
3     $argumentResolver);  
4 $response = $framework->handle($request);  
5 $response->send();
```

Listing 7.2 – Nouveau Contrôleur Frontal (web/front.php)

Nous avons maintenant une architecture propre avec une séparation claire entre le framework réutilisable (`Simplex`) et l'application spécifique (`Calendar`).

7.4 Cycle de Vie de la Requête

1. `front.php` instancie les composants (Matcher, Resolvers) et crée l'objet `Simplex`.
2. Il appelle `$framework->handle($request)`.
3. **Dans le Framework :**
 - (a) Le Matcher analyse la requête.
 - (b) Les Resolvers préparent le contrôleur et ses arguments.

-
- (c) Le contrôleur est exécuté et retourne une réponse.
 - (d) Le Framework retourne cette réponse à `front.php`.
4. `front.php` envoie la réponse.

Chapitre 8

Les Tests Unitaires

Pour garantir la fiabilité de notre framework, nous avons mis en place des tests unitaires avec PHPUnit.

8.1 Installation et Configuration

Nous avons installé PHPUnit via Composer :

```
1 composer require --dev phpunit/phpunit
```

Et créé un fichier de configuration `phpunit.dist.xml` standard.

8.2 Refactoring pour la Testabilité

Pour pouvoir tester notre classe `Framework` isolément, nous l'avons modifiée pour qu'elle dépende d'interfaces plutôt que de classes concrètes. Cela nous permet d'utiliser des "Mocks" (doublures de test).

```
1 use Symfony\Component\HttpKernel\Controller\  
    ArgumentResolverInterface;  
2 use Symfony\Component\HttpKernel\Controller\  
    ControllerResolverInterface;  
3 use Symfony\Component\Routing\Matcher\UrlMatcherInterface;  
4  
5 class Framework  
6 {  
7     public function __construct(  
8         private UrlMatcherInterface $matcher,  
9         private ControllerResolverInterface $controllerResolver,  
10        private ArgumentResolverInterface $argumentResolver,  
11    ) {  
12    }  
13    // ...
```

Listing 8.1 – Refactoring de Framework (`src/Simplex/Framework.php`)

8.2.1 Pourquoi utiliser des Interfaces ?

L'utilisation d'interfaces est une bonne pratique essentielle pour les tests unitaires. Elle nous permet de respecter le principe d'Inversion de Dépendance (SOLID) et facilite grandement le *Mocking* :

- **Isolation** : En dépendant d'interfaces, notre classe **Framework** ne dépend plus des implémentations concrètes (souvent complexes) de Symfony. On teste uniquement notre logique, pas celle des composants externes.
- **Mocking Facile** : PHPUnit peut créer très facilement des "Mocks" (doublures) à partir d'interfaces. Si nous utilisions des classes concrètes, nous serions limités (notamment si ces classes étaient **final**, ce qui est courant dans le code moderne).
- **Contrat Clair** : L'interface définit un contrat strict. Peu importe l'objet réel passé (un vrai routeur ou un mock), tant qu'il respecte le contrat, notre test est valide.

8.3 Nos premiers tests (FrameworkTest)

Nous avons écrit une classe de test `tests/Simplex/Tests/FrameworkTest.php` pour vérifier le comportement du framework dans différentes situations :

- **404 Not Found** : Lorsque le `UrlMatcher` lance une `ResourceNotFoundException`.
- **500 Error** : Lorsqu'une exception générique survient.
- **200 OK** : Lorsqu'une route est trouvée et qu'un contrôleur retourne une réponse valide.

L'exécution des tests confirme que notre framework se comporte comme attendu :

```
1 $ ./vendor/bin/phpunit
2 OK (3 tests, 9 assertions)
```

Nous pouvons maintenant continuer le développement en toute confiance !

Chapitre 9

Le Composant EventDispatcher

Notre framework est fonctionnel, mais il manque d'extensibilité. Pour permettre aux développeurs d'agir sur le cycle de vie de la requête (Hook), nous intégrons le composant EventDispatcher.

9.1 Installation

```
1 composer require symfony/event-dispatcher
```

9.2 Mise en place des événements

Nous avons créé une classe `ResponseEvent` pour transporter les objets `Request` et `Response` :

```
1 class ResponseEvent extends Event {  
2     // ... __construct(Response $response, Request $request)  
3 }
```

Listing 9.1 – La classe `ResponseEvent` (`src/Simplex/ResponseEvent.php`)

Et modifié la classe `Framework` pour dispatcher cet événement juste avant de renvoyer la réponse :

```
1 // src/Simplex/Framework.php  
2 $this->dispatcher->dispatch(new ResponseEvent($response, $request),  
3     'response');  
return $response;
```

Listing 9.2 – Dispatch de l'événement `response`

9.3 Création de Subscribers

Nous avons créé deux "Subscribers" pour démontrer l'extensibilité :

1. **GoogleListener** : Ajoute un code de tracking ("GA CODE") dans le corps de la réponse si c'est du HTML. 2. **ContentLengthListener** : Ajoute automatiquement l'en-tête `Content-Length`.

Voici le code du `GoogleListener` avec les explications :


```

1 class GoogleListener implements EventSubscriberInterface
2 {
3     // called auto when response event is dispatched
4     public function onResponse(ResponseEvent $event): void
5     {
6         $response = $event->getResponse();
7
8         if ($response->isRedirection()
9             || ($response->headers->has('Content-Type') && !
10                str_contains($response->headers->get('Content-Type'), 'html'))
11             || 'html' !== $event->getRequest()->getRequestFormat()
12         ) {
13             return;
14         }
15
16         $response->setContent($response->getContent(). 'GA CODE');
17     }
18
19     // Tell dispatcher: "When 'response' happens, call 'onResponse'
20     , "
21     public static function getSubscribedEvents(): array
22     {
23         return ['response' => 'onResponse'];
24     }
25 }

```

Listing 9.3 – GoogleListener (src/Simplex/GoogleListener.php)

L'enregistrement se fait dans le contrôleur frontal :

```

1 $dispatcher = new EventDispatcher();
2 $dispatcher->addSubscriber(new Simplex\ContentLengthListener());
3 $dispatcher->addSubscriber(new Simplex\GoogleListener());
4
5 $framework = new Simplex\Framework($dispatcher, ...);

```

Listing 9.4 – Enregistrement des Subscribers (web/front.php)

Grâce à ce système, nous pouvons désormais modifier le comportement du framework sans toucher à son cœur !

9.4 Cycle de Vie de la Requête (Final)

1. `front.php` instancie le `EventDispatcher` et y attache les Subscribers.
2. `Simplex` est instancié avec ce Dispatcher.
3. `$framework->handle($request)` est appelé.
4. Le contrôleur est exécuté et retourne une réponse (ex : HTML brut).
5. **Nouveau** : Le Framework dispatche l'événement `'response'`.
6. **Nouveau** : `GoogleListener` intercepte l'événement et ajoute "GA CODE".

-
7. **Nouveau** : `ContentLengthListener` ajoute l'en-tête `Content-Length`.
 8. La réponse modifiée est retournée et envoyée au client.

Chapitre 10

Le Composant HttpKernel (HttpKernelInterface)

L'étape 10 est cruciale pour l'interopérabilité. Nous allons faire implémenter l'interface `HttpKernelInterface` à notre framework.

10.1 Implémentation de l'Interface

```
1 use Symfony\Component\HttpKernel\HttpKernelInterface;
2
3 class Framework implements HttpKernelInterface
4 {
5     public function handle(
6         Request $request,
7         int $type = HttpKernelInterface::MAIN_REQUEST,
8         bool $catch = true
9     ): Response
10    {
11        // ...
12    }
13 }
```

Listing 10.1 – Implémentation de `HttpKernelInterface` (`src/Simplex/Framework.php`)

10.2 Le Cache HTTP

En respectant ce contrat, nous pouvons désormais utiliser le `HttpCache` de Symfony, qui est un reverse proxy écrit en PHP (similaire à Varnish). Il suffit d'envelopper notre framework :

```
1 $framework = new Simplex\Framework($dispatcher, ...);
2 $framework = new HttpKernel\HttpCache\HttpCache(
3     $framework,
4     new HttpKernel\HttpCache\Store(__DIR__ . '/../cache')
5 );
6 $response = $framework->handle($request);
```

Listing 10.2 – Ajout du Cache (`web/front.php`)

10.3 Test du Cache

Pour vérifier, nous avons ajouté un nombre aléatoire et un temps de cache (TTL) de 10 secondes dans notre contrôleur :

```
1 $response = new Response('Random: ' . rand());  
2 $response->setTtl(10);
```

Lors de nos tests, le nombre aléatoire reste identique pendant 10 secondes, prouvant que le cache fonctionne !

10.4 Cycle de Vie de la Requête

1. `front.php` instancie `Simplex`.
2. Il instancie `HttpCache` en lui passant le framework.
3. Il appelle `$httpCache->handle($request)`.
4. **Dans `HttpCache` :**
 - (a) Vérifie si une réponse valide est en cache.
 - (b) **Si OUI** : Retourne la réponse en cache (Framework non appelé).
 - (c) **Si NON** : Appelle `$framework->handle()`.
 - (d) Stocke la réponse du framework dans le cache (selon les headers).
5. La réponse est envoyée au client.

Chapitre 11

Le Composant HttpKernel (La Classe HttpKernel)

Enfin, l'étape 11 consiste à remplacer notre implémentation manuelle du framework par la classe standard **Symfony**. Cette classe est le cœur de Symfony et gère tout le cycle de vie de manière robuste.

11.1 Le Framework Simplifié

Notre classe **Framework** n'a plus besoin de code, elle hérite simplement de **HttpKernel** :

```
1 use Symfony\Component\HttpKernel\HttpKernel;
2
3 class Framework extends HttpKernel
4 {
5 }
```

Listing 11.1 – Héritage de HttpKernel (src/Simplex/Framework.php)

11.2 Mise à jour du Contrôleur Frontal

Dans `web/front.php`, nousinstancions désormais le **Dispatcher** avec des souscripteurs standards de Symfony, comme le **RouterListener**, qui remplace notre logique de routage manuelle :

```
1 $dispatcher->addSubscriber(new HttpKernel\EventListener\
    RouterListener($matcher, $requestStack));
```

11.3 Gestion des Erreurs (ErrorListener)

Nous avons également ajouté un **ErrorListener** pour gérer les exceptions de manière élégante. Au lieu de laisser le framework planter, nous déléguons la gestion à un **ErrorController**.

Dans `web/front.php` :

```
1 $dispatcher->addSubscriber(new HttpKernel\EventListener\
    ErrorListener(
```

```
2         'Calendar\Controller\ErrorController::exception'  
3     ));
```

Et le contrôleur :

```
1 public function exception(FlattenException $exception): Response  
2 {  
3     $msg = 'Something went wrong! ('. $exception->getMessage(). ')';  
4     return new Response($msg, $exception->getStatusCode());  
5 }
```

Listing 11.2 – ErrorController (src/Calendar/Controller/ErrorController.php)

11.4 Préparation de la Réponse (ResponseListener)

Comme recommandé par la documentation, nous avons ajouté le `ResponseListener` pour garantir que nos réponses sont conformes au protocole HTTP (encodage, etc.) :

```
1 $dispatcher->addSubscriber(new HttpKernel\EventListener\  
    ResponseListener('UTF-8'));
```

11.5 Le StringResponseListener

Pour montrer la puissance des événements, nous avons ajouté un `StringResponseListener` qui permet aux contrôleurs de retourner de simples chaînes de caractères (converties automatiquement en `Response`).

```
1 public function onView(ViewEvent $event): void  
2 {  
3     $response = $event->getControllerResult();  
4     if (is_string($response)) {  
5         $event->setResponse(new Response($response));  
6     }  
7 }
```

Listing 11.3 – StringResponseListener (src/Simplex/StringResponseListener.php)

Cela nous permet d'écrire des contrôleurs plus simples :

```
1 public function index(int $year)  
2 {  
3     return 'Yep, this is a leap year!'; // Retourne une string !  
4 }
```

11.6 Cycle de Vie de la Requête (Final)

Avec `HttpKernel`, le cycle est complet et standardisé. La plupart de la logique est maintenant encapsulée dans la classe `HttpKernel` du dossier `vendor`, et nous intervenons via les événements :

-
1. `front.php` appelle `HttpKernel->handle()`.
 2. **kernel.request** : Dans `HttpKernel::handleRaw()`, l'événement est dispatché. Le `RouterListener` (enregistré dans `front.php`) trouve la route et définit `_controller`.
 3. **kernel.controller** : Toujours dans `handleRaw()`, le résolveur instancie le contrôleur.
 4. **kernel.controller_arguments** : Le résolveur d'arguments prépare les paramètres.
 5. **Exécution du Contrôleur** : Le code du contrôleur est exécuté via `$controller(...$arguments)`.
 6. **kernel.view** : Si le contrôleur ne retourne pas de `Response`, notre `StringResponseListener` intervient pour convertir la string.
 7. **kernel.response** : Dans `HttpKernel::filterResponse()`, l'événement est dispatché. `ResponseListener`, `GoogleListener` et `ContentLengthListener` modifient la réponse.
 8. **kernel.finish_request** : Tâches de nettoyage dans `HttpKernel::finishRequest()`.
 9. **Envoi** : Dans `front.php`, `$response->send()` envoie la réponse finale.

Chapitre 12

L'Injection de Dépendances

Pour cette dernière étape, nous allons résoudre un problème majeur de notre architecture actuelle : la complexité de `web/front.php` et le manque de flexibilité dans la configuration de nos objets.

Nous allons introduire le composant `symfony/dependency-injection` pour centraliser la création et la configuration de tous nos objets (services).

12.1 Installation

```
1 composer require symfony/dependency-injection
```

12.2 Configuration du Conteneur (`src/container.php`)

Nous avons créé un fichier `src/container.php` qui configure le `ContainerBuilder`. Ce fichier définit :

- Les paramètres (ex : le charset).
- Les services du noyau (Matcher, Resolvers, RequestStack).
- Les écouteurs d'événements (`RouterListener`, `ErrorListener`, nos listeners personnalisés).
- Le `EventDispatcher` auquel on injecte les écouteurs via `addMethodCall`.
- Et enfin, notre classe `Framework` qui reçoit toutes ses dépendances.

L'avantage est que l'instanciation des objets est retardée (Lazy Loading) et que nous pouvons changer la configuration (ex : charset) sans toucher au code des classes.

12.3 Le Contrôleur Frontal Nettoyé

Grâce au conteneur, `web/front.php` redevient extrêmement simple :

```
1 $container = include __DIR__.'../src/container.php';
2 $container->setParameter('debug', true);
3 $container->setParameter('charset', 'UTF-8');
4 $container->set('routes', $routes);
5
6 $response = $container->get('framework')->handle($request);
7 $response->send();
```

Listing 12.1 – Front Controller avec ID (web/front.php)

Nous avons injecté les routes dynamiquement et récupéré notre framework entièrement configuré en une seule ligne !

Conclusion

Nous sommes partis d'un simple script PHP "Hello World" et nous avons reconstruit, étape par étape, un framework MVC moderne, robuste et testable, utilisant les mêmes composants que Symfony.

Nous avons appris à maîtriser :

- **HttpFoundation** pour abstraire les requêtes/réponses.
- **Routing** pour mapper les URL.
- **HttpKernel** (ControllerResolver) pour exécuter la logique.
- **EventDispatcher** pour rendre le système extensible.
- **HttpKernel** (la classe) pour le cycle de vie standard.
- **DependencyInjection** pour l'organisation et la configuration.

Ce framework "Simplex" est une base solide pour comprendre comment fonctionnent les frameworks PHP modernes.